SwiftX

FinTech Financial Inclusion Platform

TEST SCENARIOS AND CASES

Test-Driven Process | SonarQube Quality Strategy

Software Engineering — Mini Project
Project Duration: 6-8 Weeks
Submitted by: SwiftX Development Team
May 2026

1. Introduction

SwiftX is a mobile-first FinTech platform designed to provide zero-fee remittance services, secure multi-currency digital wallets, and AI-powered financial guidance to migrant workers and underserved populations. The system is built on Next.js 14, Supabase (PostgreSQL with Row Level Security), Anthropic Claude AI, and a SHA-256 hash-chained ledger with optional Polygon Amoy blockchain anchoring.

This document defines the comprehensive test scenarios and test cases for SwiftX. Testing is structured around the Test-Driven Development (TDD) methodology — Red, Green, Refactor — extended with risk-aware tagging to ensure every identified project risk has at least one automated check watching over it. SonarQube serves as the central quality gate, inspecting each change for bugs, security vulnerabilities, hotspots, code smells, and coverage gaps before code can be merged.

Ten top-level test scenarios are defined, each decomposed into detailed test cases that span functional correctness, security, performance, integration, and compliance dimensions. Each test case is tagged with the risk ID it mitigates, consistent with the Assignment 4 Risk Mitigation Plan.

2. Testing Approach

2.1 Test-Driven Development (TDD) Cycle

All SwiftX test cases follow the classic Red–Green–Refactor cycle:

1. RED — Write a failing test describing the expected behaviour before any production code exists.
2. GREEN — Write the minimal production code that makes the test pass without breaking other tests.
3. REFACTOR — Clean the code while keeping all tests green. SonarQube is run after every refactor step.

2.2 SonarQube Quality Gate

Every pull request is blocked unless it passes the SonarQube Quality Gate:

- No new bugs of any severity
- No new High or Critical security vulnerabilities
- New code coverage ≥ 80%
- New code duplication ratio ≤ 3%
- All security hotspots reviewed

Full SonarQube reports are automatically archived to an Amazon S3 bucket after every CI run for compliance evidence and historical trend analysis.

2.3 Risk-to-Test Mapping

Each test case in this document is tagged with the risk it mitigates (R-01 through R-12) from the Assignment 4 Risk Register. The table below shows coverage at a glance:

Risk ID	Description	Covered By	Severity
R-01	AI/ML model accuracy drift	TS-01, TS-06	High
R-02	Bias detection false negatives	TS-01, TS-06	High
R-03	External system integration failures	TS-04, TS-08	High
R-04	PII exposure and security lapses	TS-05, TS-07	Critical
R-05	Performance degradation under load	TS-09	Medium
R-06	Scope creep	Process control	Low
R-07	Third-party dependency outage	TS-04, TS-08	Medium
R-08	Over-reliance on AI outputs	TS-06	Low
R-09	Key team member unavailable	Process control	Low
R-10	Document parsing failures	TS-05	Medium
R-11	Compute cost overrun	TS-09	Low
R-12	Low user adoption	TS-10	Low

3. Test Scenarios Overview

Ten top-level test scenarios cover the complete SwiftX feature surface:

Scenario ID	Scenario Title	Test Types	Cases
TS-01	User Registration, Authentication & KYC	Functional, Security	18 Cases
TS-02	Multi-Currency Wallet Management	Functional	12 Cases
TS-03	Zero-Fee Remittance Transfer Flow	Functional, Integration	20 Cases
TS-04	FX Rate Fetching and Caching	Functional, Integration	10 Cases
TS-05	Hash-Chain Ledger Integrity	Security, Data Integrity	12 Cases
TS-06	AI-Powered Financial Insights	Functional, AI Quality	10 Cases
TS-07	Security, PII Protection & RLS	Security, Compliance	16 Cases
TS-08	Third-Party Integration & Resilience	Integration, Resilience	10 Cases
TS-09	Performance and Scalability	Non-Functional	8 Cases
TS-10	Usability, Accessibility & Localisation	Non-Functional, UX	8 Cases

4. Detailed Test Cases

TS-01 — User Registration, Authentication & KYC

Covers user sign-up via magic link / phone OTP, KYC document upload and status transitions (pending → verified → rejected), session management, and re-authentication requirements.

TC ID	Test Description	Preconditions	Test Steps	Expected Result	Priority	Risk Tag
TC-01-01	Successful new user sign-up with magic link	Email not previously registered	1. Enter valid email. 2. Submit form. 3. Click magic link in inbox. 4. Confirm redirect to wallet home.	User profile created; wallet home displayed; KYC status = 'pending'.	**High**	R-04
TC-01-02	Sign-up with already-registered email	Email already exists in auth.users	1. Enter duplicate email. 2. Submit.	Error: 'An account with this email already exists'. No duplicate profile row created.	**High**	R-04
TC-01-03	Phone OTP — correct code	Valid phone number, OTP sent	1. Enter phone. 2. Enter correct OTP within 5 min.	Session token issued; user authenticated.	**High**	R-04
TC-01-04	Phone OTP — expired code	OTP older than 5 minutes	1. Enter phone. 2. Wait 6 min. 3. Enter original OTP.	Error: 'OTP has expired'. New OTP prompt shown.	**High**	R-04
TC-01-05	KYC document upload — accepted	User logged in; KYC status = 'pending'	1. Upload valid passport image (< 5 MB). 2. Submit.	Document stored securely; KYC status transitions to 'verified'.	**High**	R-04
TC-01-06	KYC document upload — oversized file	User logged in	1. Attempt to upload 12 MB image.	Error: 'File exceeds 5 MB limit'. Status remains 'pending'.	**Medium**	R-10
TC-01-07	KYC document rejected by admin	Document under review	1. Admin marks document as rejected via dashboard.	KYC status = 'rejected'; user notified; transfer capability suspended.	**High**	R-04
TC-01-08	Access protected route without authentication	Unauthenticated browser session	1. Navigate directly to /send.	Redirect to /login; no wallet data exposed.	**High**	R-04
TC-01-09	Session token expiry — forced re-login	Valid session older than configured TTL	1. Let session expire. 2. Attempt to initiate transfer.	401 returned; user redirected to login; no partial transfer persisted.	**High**	R-04
TC-01-10	Brute-force OTP — 5 failed attempts	Valid phone registered	1. Enter wrong OTP 5 times in succession.	Account temporarily locked; 'Too many attempts' error; lockout duration shown.	**High**	R-04

TS-02 — Multi-Currency Wallet Management

Covers wallet creation, balance display, top-up (mocked), and withdrawal (mocked via UPI/IMPS). Each user may hold wallets in USD, AED, and INR simultaneously.

TC ID	Test Description	Preconditions	Test Steps	Expected Result	Priority	Risk Tag
TC-02-01	Auto-create USD wallet on first login	New user, country_code = 'AE' (Dubai)	1. Complete registration. 2. Observe wallet home.	USD wallet created automatically; balance = 0.0000.	**High**	R-03
TC-02-02	Display correct multi-currency balances	User holds USD and INR wallets	1. Navigate to wallet home.	Both currencies shown with correct balances to 4 decimal places.	**High**	R-01
TC-02-03	Top-up wallet (mocked rail)	USD wallet at $0	1. Select Top Up. 2. Enter $500. 3. Confirm.	Balance increases to $500.0000; transaction record type='topup' created.	**High**	R-03
TC-02-04	Top-up with negative amount	User on top-up screen	1. Enter -100. 2. Submit.	Validation error: 'Amount must be positive'. No transaction created.	**High**	R-01
TC-02-05	Top-up with zero amount	User on top-up screen	1. Enter 0. 2. Submit.	Validation error: 'Amount must be greater than zero'.	**Medium**	R-01
TC-02-06	Withdraw from wallet (mocked UPI)	INR wallet balance ≥ withdrawal amount	1. Select Withdraw. 2. Enter ₹5,000. 3. Confirm.	Balance decreases; transaction type='withdraw' recorded with hash.	**High**	R-03
TC-02-07	Withdraw more than available balance	INR wallet balance = ₹2,000	1. Attempt to withdraw ₹5,000.	Error: 'Insufficient balance'. No transaction created.	**High**	R-01
TC-02-08	Auto-create INR wallet for recipient	Recipient exists but has no INR wallet	1. Initiate transfer to recipient's phone.	INR wallet auto-created for recipient; transfer proceeds.	**High**	R-03

TS-03 — Zero-Fee Remittance Transfer Flow

The core business flow: Ravi (Dubai, USD) sends to his mother (Kerala, INR). Tests cover the happy path, edge cases, AML blocking, hash chain generation, and atomic rollback on failure.

TC ID	Test Description	Preconditions	Test Steps	Expected Result	Priority	Risk Tag
TC-03-01	Successful USD → INR transfer	Sender: $500 USD. Recipient: registered. KYC: verified.	1. Enter ₹15,000 target. 2. Verify FX breakdown. 3. Confirm.	Debit and credit applied atomically; hash generated; realtime update to recipient.	**High**	R-01
TC-03-02	FX breakdown screen shows correct amounts	Live FX rate available	1. Enter ₹15,000. 2. Inspect FX breakdown screen.	USD payable, 0% fee line, INR received, live rate all displayed correctly.	**High**	R-01
TC-03-03	Transfer fee is exactly $0.00	Any valid transfer scenario	1. Initiate any transfer. 2. Check fee line on breakdown screen.	Fee always shows $0.00 regardless of amount.	**High**	R-01
TC-03-04	Transfer with insufficient balance	Sender USD balance = $10	1. Attempt $500 transfer.	Error: 'Insufficient balance'. No debit or credit occurs.	**High**	R-01
TC-03-05	Transfer to unregistered phone number	Phone not in profiles table	1. Enter unknown phone number.	Error: 'Recipient not found'. No transaction.	**Medium**	R-03
TC-03-06	AML flag — amount over threshold ($10,000)	Sender balance ≥ $10,000	1. Attempt $11,000 single transfer.	Transaction flagged in aml_flags; severity='high'; transfer placed in 'pending' for review.	**High**	R-04
TC-03-07	AML flag — velocity check (5 transfers in 1 hour)	Sender completes 4 transfers within 60 min	1. Initiate 5th transfer.	5th transaction flagged; rule_triggered='velocity_check'; admin notified.	**High**	R-04
TC-03-08	Database RPC failure — atomic rollback	Mocked: Supabase RPC returns error mid-transfer	1. Initiate transfer. 2. Simulate DB error.	No balance change; no transaction record created; error surfaced to user.	**High**	R-03
TC-03-09	Concurrent transfers — no double-spend	Two simultaneous transfer requests from same wallet	1. Fire two identical requests within 50 ms.	Only one succeeds; second rejected with 'Insufficient balance'.	**High**	R-01
TC-03-10	Hash generated and linked after transfer	Successful transfer completed	1. Query transaction row.	current_hash is non-null SHA-256 hex; prev_hash references the preceding transaction's hash.	**High**	R-05

TS-04 — FX Rate Fetching and Caching

Validates the FX rate pipeline: cache hit, cache miss (live fetch), cache TTL expiry, fallback behaviour, and the applied margin disclosure.

TC ID	Test Description	Preconditions	Test Steps	Expected Result	Priority	Risk Tag
TC-04-01	Cache hit — FX rate returned within TTL	Rate cached < 60 minutes ago	1. Request USD→INR. 2. Inspect HTTP calls.	Rate returned from fx_rates_cache; no call to exchangerate.host.	**High**	R-07
TC-04-02	Cache miss — live rate fetched and stored	No cache entry or entry > 60 min old	1. Request USD→INR with empty cache.	Rate fetched from exchangerate.host; stored with 0.2% margin applied; cached.	**High**	R-07
TC-04-03	FX API unavailable — fallback rate used	Mocked: exchangerate.host returns 503	1. Request rate with external API down.	Hardcoded fallback rate used; user sees 'Estimated rate' disclaimer.	**High**	R-07
TC-04-04	FX margin of 0.2% correctly applied	Known live rate: 1 USD = 83.50 INR	1. Fetch and inspect stored rate.	Stored rate = 83.50 × 0.998 = 83.33 INR; shown transparently on FX breakdown screen.	**High**	R-01
TC-04-05	FX rate refresh after TTL expiry	Cache entry exactly 61 minutes old	1. Request rate.	New live rate fetched; old cache entry superseded; new entry stored.	**Medium**	R-07

TS-05 — Hash-Chain Ledger Integrity

Validates the SHA-256 hash-chain logic: genesis block, chain continuity, tamper detection, Polygon anchor creation, and the chain verifier endpoint.

TC ID	Test Description	Preconditions	Test Steps	Expected Result	Priority	Risk Tag
TC-05-01	Genesis transaction hash correctly formed	First transaction ever created	1. Trigger first transfer. 2. Query transaction row.	prev_hash = 'GENESIS'; current_hash = SHA-256(row data + 'GENESIS').	**High**	R-05
TC-05-02	Subsequent transaction links to previous hash	At least one transaction exists	1. Complete second transfer. 2. Inspect prev_hash.	prev_hash equals current_hash of the immediately preceding transaction.	**High**	R-05
TC-05-03	verifyChain returns valid=true for untampered chain	5 sequential transactions committed	1. Call GET /api/verify-chain.	Response: { valid: true, brokenAt: null }.	**High**	R-05
TC-05-04	verifyChain detects tampered hash	Transaction row hash manually altered in DB	1. Corrupt current_hash of row 3. 2. Call GET /api/verify-chain.	Response: { valid: false, brokenAt: 3 }.	**High**	R-04

TC ID	Test Description	Preconditions	Test Steps	Expected Result	Priority	Risk Tag
TC-05-05	Polygon anchor created every 5 transactions	Anchor cron job enabled	1. Complete 5 transfers. 2. Wait for cron. 3. Query polygon_anchors.	New row in polygon_anchors; polygon_tx_hash non-null; from_tx_id and to_tx_id set.	**High**	R-05
TC-05-06	Hash chain viewer shows correct hash on UI	Transfer completed	1. Navigate to history/[id]. 2. Inspect hash chain viewer.	current_hash and prev_hash displayed; PolygonScan link present if anchored.	**Medium**	R-05

TS-06 — AI-Powered Financial Insights (Claude API)

Tests the Anthropic Claude integration: prompt construction, multilingual output, token limits, graceful degradation when the API is unavailable, and relevance of generated advice.

TC ID	Test Description	Preconditions	Test Steps	Expected Result	Priority	Risk Tag
TC-06-01	Insight generated in English	User language = 'en'; ≥ 1 transaction exists	1. Navigate to /insights.	Insight card displays English text; ≤ 50 words; references user's transaction data.	**High**	R-01
TC-06-02	Insight generated in Hindi	User language = 'hi'	1. Switch language to Hindi. 2. Navigate to /insights.	Insight card displays Devanagari Hindi text; contextually relevant.	**High**	R-02
TC-06-03	Insight does not exceed 50-word limit	Any valid user session	1. Generate multiple insights. 2. Count words in each.	All generated insights ≤ 50 words.	**Medium**	R-08
TC-06-04	Claude API unavailable — graceful degradation	Mocked: Anthropic API returns 503	1. Navigate to /insights with API down.	Fallback message shown: 'Insights temporarily unavailable'; no app crash.	**High**	R-07
TC-06-05	Insight references real transaction numbers	User has sent $1,200 this quarter	1. Generate insight.	Dollar amount or frequency mentioned in insight; not generic filler.	**Medium**	R-01
TC-06-06	Micro-investment SIP prompt from insight	Insight card generated	1. Click 'Start ₹100 SIP' CTA from insight.	User navigated to /invest; SIP amount pre-filled as ₹100.	**Medium**	R-12

TS-07 — Security, PII Protection & Row Level Security

Validates Supabase RLS policies, data isolation between users, protection against SQL injection and XSS vectors, encrypted storage of KYC documents, and SonarQube vulnerability gate enforcement.

TC ID	Test Description	Preconditions	Test Steps	Expected Result	Priority	Risk Tag
TC-07-01	User A cannot read User B's wallets	Two registered users; User A authenticated	1. Authenticate as User A. 2. Query wallets table directly.	Only User A's wallets returned; no User B rows visible.	**High**	R-04
TC-07-02	User A cannot read User B's transactions	Both users have transactions	1. As User A, query transactions table.	Only transactions involving User A's wallet IDs returned.	**High**	R-04
TC-07-03	Admin role sees all transactions	User with country_code='ADMIN'	1. Authenticate as admin. 2. Query transactions.	All transaction rows returned; compliance dashboard populated.	**High**	R-04
TC-07-04	SQL injection in transfer amount field	Authenticated user on send page	1. Enter "100; DROP TABLE wallets;" in amount field. 2. Submit.	Input rejected by parameterised query; no DB mutation; error shown.	**High**	R-04
TC-07-05	XSS payload in full_name field	Registration form	1. Enter <script>alert('xss')</script> as full_name.	Input sanitised; script not executed; stored as escaped string.	**High**	R-04
TC-07-06	KYC document stored with server-side encryption	Valid KYC upload	1. Upload document. 2. Check Supabase Storage bucket settings.	Document stored with SSE enabled; direct public URL returns 403.	**High**	R-04
TC-07-07	Hard-coded secret detected by SonarQube	Developer accidentally commits API key in source	1. Push code with hard-coded ANTHROPIC_API_KEY.	SonarQube flags vulnerability; Quality Gate fails; merge blocked.	**High**	R-04
TC-07-08	Unauthenticated POST to /api/transfer blocked	No auth token in request headers	1. Send POST /api/transfer without Authorization header.	401 Unauthorized returned; no transfer executed.	**High**	R-04

TS-08 — Third-Party Integration & Resilience

Tests the behaviour of SwiftX when dependent services (Supabase, Anthropic Claude, exchangerate.host, Polygon Amoy, Vercel/AWS) are degraded or unavailable.

TC ID	Test Description	Preconditions	Test Steps	Expected Result	Priority	Risk Tag
TC-08-01	Supabase DB connection lost during transfer	Mocked: DB connection dropped after debit, before credit	1. Initiate transfer. 2. Simulate connection drop.	Atomic rollback via RPC; both wallets unchanged; user sees error.	**High**	R-03
TC-08-02	Supabase Auth outage — graceful degradation	Mocked: Auth endpoint returns 503	1. Attempt login.	Error: 'Authentication service temporarily unavailable'; retry prompt.	**High**	R-03
TC-08-03	exchangerate.host rate-limited (429)	Mocked: API returns 429	1. Request FX rate.	Cached rate used (if available) or fallback; 'Estimated rate' label shown.	**High**	R-07
TC-08-04	Polygon Amoy anchor cron — network timeout	Mocked: Polygon RPC times out	1. Trigger anchor cron.	Anchor skipped; retry scheduled; no exception propagates to user flow.	**Medium**	R-07
TC-08-05	Vercel CDN edge cache serves stale UI	Mocked: Vercel revalidation delayed	1. Navigate to app after deployment.	User sees version indicator; hard-refresh prompt if stale build detected.	**Low**	R-07

TS-09 — Performance and Scalability

Non-functional tests validating response times under normal and peak load, database query efficiency, and the PWA offline experience. SonarQube Code Smells and Hotspots complement these with static analysis of known performance anti-patterns.

TC ID	Test Description	Preconditions	Test Steps	Expected Result	Priority	Risk Tag
TC-09-01	Transfer API responds within 2 seconds at p95	50 concurrent users	1. Run load test with k6; 50 VUs for 60 seconds.	p95 transfer response time ≤ 2,000 ms; error rate < 0.1%.	**High**	R-05
TC-09-02	FX rate endpoint responds within 200 ms (cache hit)	Rate pre-cached	1. Send 100 requests to GET /api/fx.	All responses ≤ 200 ms; no DB query executed.	**Medium**	R-05

TC ID	Test Description	Preconditions	Test Steps	Expected Result	Priority	Risk Tag
TC-09-03	Wallet home page loads within 1 second on 4G	Mobile device, 4G throttled (Chrome DevTools)	1. Navigate to wallet home. 2. Measure LCP.	Largest Contentful Paint ≤ 1,000 ms on simulated 4G.	**High**	R-12
TC-09-04	500 concurrent users — no data corruption	Load test environment	1. k6 script fires 500 simultaneous transfer requests.	All wallet balances correct; no phantom transactions; no duplicate hashes.	**High**	R-05
TC-09-05	SonarQube detects N+1 query code smell	Developer writes loop calling DB per row	1. Push code with N+1 pattern.	SonarQube flags Code Smell; developer prompted to batch query.	**Medium**	R-05

TS-10 — Usability, Accessibility & Localisation

Tests multilingual UI, voice input, PWA installability, and WCAG 2.1 AA accessibility. These tests directly address the low user-adoption risk (R-12) and the multilingual inclusion mandate.

TC ID	Test Description	Preconditions	Test Steps	Expected Result	Priority	Risk Tag
TC-10-01	Language toggle switches UI to Hindi	User authenticated; default language = 'en'	1. Tap language toggle. 2. Select Hindi.	All labels, buttons, and AI insight shown in Devanagari Hindi immediately.	**High**	R-12
TC-10-02	Language preference persisted across sessions	User has set language = 'hi'	1. Log out. 2. Log back in.	App launches in Hindi without requiring re-selection.	**Medium**	R-12
TC-10-03	PWA installable on Android Chrome	Android device with Chrome	1. Open demo URL. 2. Tap 'Add to Home Screen'.	App installs successfully; launches without browser chrome; looks native.	**High**	R-12
TC-10-04	Voice command initiates send flow	User on home screen; microphone permission granted	1. Tap microphone icon. 2. Say 'Send money'.	App navigates to /send; amount field focused.	**Medium**	R-12
TC-10-05	Screen reader announces transfer confirmation	Accessibility tool (NVDA or VoiceOver) active	1. Complete transfer. 2. Listen to announcement.	Confirmation amount and recipient name announced correctly; no orphaned ARIA roles.	**High**	R-12

TC ID	Test Description	Preconditions	Test Steps	Expected Result	Priority	Risk Tag
TC-10-06	Keyboard-only navigation completes full send flow	Desktop browser	1. Use Tab/Enter only to navigate and submit transfer.	All interactive elements reachable; focus indicators visible; transfer completes.	**Medium**	R-12

5. TDD Walk-Through — Transfer Flow (TC-03-01)

To illustrate how TDD and SonarQube interact in practice, the following walk-through shows TC-03-01 applied to the core transfer API route:

Step 1 — RED

The developer writes a Jest test against /api/transfer with a valid payload (fromUserId, toUserPhone, fromCurrency='USD', toCurrency='INR', amount=180.65). The test asserts a 200 response and checks that from_wallet balance has decreased and to_wallet balance has increased by the correct converted amount. The test fails immediately because the route returns 404.

Step 2 — GREEN

The developer implements executeTransfer() in lib/transfer.ts — fetching FX rate, resolving wallets, running AML checks, calling the Supabase RPC, and computing the hash. The Jest test passes. SonarQube is triggered and flags one Security Hotspot: user input not validated before being passed to the RPC. The developer adds a Zod schema validation layer.

Step 3 — REFACTOR

The monolithic function is split into getFxRate(), runAmlChecks(), and executeTransfer(). Variable names are improved. After each refactor, all tests are run. SonarQube's Quality Gate passes: 0 new bugs, 0 high vulnerabilities, coverage on new code = 87%, duplication = 0%.

Step 4 — EXTEND

TC-03-04 (insufficient balance) and TC-03-09 (concurrent double-spend) are added as additional Red–Green–Refactor cycles. Each is tagged R-01. The risk dashboard now shows 3 automated checks watching over the transfer accuracy risk.

6. SonarQube Quality Gate Configuration

The following Quality Gate conditions apply to every pull request on the SwiftX repository. A change is blocked if any condition fails:

Condition	Threshold	Tool / Method	Risk Mitigated
New Bugs	= 0	Any severity	Functional correctness
New Vulnerabilities	= 0 (High/Critical)	Security scan	R-04 PII protection
Security Hotspots	All reviewed	Manual review required	R-04
New Code Coverage	≥ 80%	Jest + Supertest	All risks
New Duplication	≤ 3%	Clone detection	Code maintainability
Code Smells (Blocker)	= 0	Static analysis	R-05 performance

7. Report Archival Strategy

After every CI run, the full SonarQube analysis report (issue list, severity breakdown, coverage numbers, duplication metrics, and trend data) is automatically exported and pushed to an Amazon S3 bucket. The archival pipeline ensures:

- Historical trend analysis across the full 10-week project timeline — allowing the team to identify whether quality is improving or degrading sprint over sprint.
- Compliance evidence — during a security audit or academic review, the complete set of scan reports can be produced for any date range without relying on the SonarQube server remaining live.
- Team learning — past reports provide concrete examples of the issues the tool has caught, which accelerates onboarding of new contributors.
- The S3 bucket applies server-side AES-256 encryption and lifecycle rules consistent with SwiftX's broader data handling policy. Only authorised team members hold retrieval permissions.

Note on dependencies: the archival pipeline itself depends on AWS S3 availability. If S3 experiences a regional event, archival is delayed until recovery. This is an accepted risk, consistent with the broader SwiftX dependency on managed cloud providers.

8. Conclusion

This document defines ten top-level test scenarios and over 80 individual test cases covering the complete SwiftX feature surface — authentication, wallets, remittance, FX, hash-chain integrity, AI insights, security/RLS, third-party resilience, performance, and usability.

Every test case is tagged with the risk ID it mitigates from the Assignment 4 Risk Register. The combination of TDD (Red–Green–Refactor) and SonarQube's automated Quality Gate means that no change can be merged until it is functionally correct, well-covered, and free of known security patterns. Full reports are archived to S3 for audit and compliance purposes.

Within the explicit boundary that SonarQube inspects only SwiftX's own code (not the internals of Supabase, Anthropic, or AWS), this testing strategy provides a comprehensive, defensible, and fully automated assurance that SwiftX is production-ready.

— End of Assignment 5 : Test Scenarios and Cases —